SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – DESIGN TASK

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 3 – Design Task
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	A Lokanath Reddy	**Reg. No.:**	192425321
Section / Batch:	AIML	**Date:**	

Code of Conduct

I A Lokanath Reddy/192425321 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
N-gram Based Next-Word Prediction for Smart Text Input	Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation Next-word prediction	Q1
Smoothing and Backoff Model for Speech/Text Prediction	N-gram frequency calculation Unsmoothed probability estimation Backoff mechanism	Q2
Entropy-Based Evaluation of an	Training and testing corpus creation	Q3

English Language Model	Unigram/bigram/trigram construction Probability estimation	
Part-of-Speech Tagging System for Text Analysis	Text preprocessing and tokenization English/Penn Treebank tagset representation Rule-based POS tagging	Q4

Annexure A – Design Task

Task 1 – N-gram Based Text Prediction

Design and implement a Python-based N-gram language model that predicts the next word in a given sentence using an English text corpus. The system should preprocess the corpus, perform sentence segmentation and tokenization, and construct unigram, bigram, and trigram models by calculating word-frequency counts and maximum-likelihood probabilities.

For an incomplete sentence such as:

"Artificial intelligence is"

the system should identify and display the top-5 most probable next words.

The program should allow the user to select N = 1, 2, or 3, display the corresponding N-gram frequency counts and probabilities, and demonstrate the behavior of the model when an unseen N-gram is encountered. Finally, test the model using multiple incomplete sentences, calculate the prediction accuracy, and discuss the limitations of using an unsmoothed N-gram model for real-world language prediction.

1. Objective and preprocessing: The objective is to predict the next word of an incomplete sentence using unigram, bigram and trigram language models. The English corpus is first converted to lowercase, divided into sentences, and tokenized into words. Special START and END tokens can be added so that sentence boundaries are represented correctly.

2. N-gram construction: A unigram stores individual word frequencies. A bigram stores two consecutive words and a trigram stores three consecutive words. Maximum-likelihood probabilities are calculated as follows:

P(w) = Count(w) / Total words

P(w2|w1) = Count(w1,w2) / Count(w1)

P(w3|w1,w2) = Count(w1,w2,w3) / Count(w1,w2).

3. Python implementation: ```python

```
from collections import Counter
import re

corpus = [
    "Artificial intelligence is transforming technology",
    "Artificial intelligence is useful in education",
    "Machine learning is a part of artificial intelligence"
```

```
]

tokens = []
for sentence in corpus:
    tokens += re.findall(r"\b[a-z]+\b", sentence.lower())

unigram = Counter(tokens)
bigram = Counter(zip(tokens, tokens[1:]))
trigram = Counter(zip(tokens, tokens[1:], tokens[2:]))

def next_words(prefix, k=5):
    words = prefix.lower().split()
    candidates = {}
    if len(words) >= 2:
        pair = tuple(words[-2:])
        denom = sum(c for (a,b,c), c in trigram.items() if (a,b) == pair)
        for (a,b,w), count in trigram.items():
            if (a,b) == pair and denom:
                candidates[w] = count / denom
    if not candidates and words:
        w1 = words[-1]
        denom = sum(c for (a,b), c in bigram.items() if a == w1)
        for (a,b), count in bigram.items():
            if a == w1 and denom:
                candidates[b] = count / denom
    return sorted(candidates.items(), key=lambda x: x[1], reverse=True)[:k]

print("Unigram:", unigram)
print("Bigram:", bigram)
print("Trigram:", trigram)
print("Top predictions:", next_words("Artificial intelligence is"))
```

4. Example prediction: For the incomplete sentence “Artificial intelligence is”, the model searches the trigram context (artificial, intelligence). If the training corpus contains “Artificial intelligence is transforming” and “Artificial intelligence is useful”, the possible next words include “transforming” and “useful”. The exact top-5 list depends on the selected corpus.

5. Unseen N-gram behavior: In an unsmoothed model, an unseen bigram or trigram receives probability 0. Therefore, if the required context was never observed in the training corpus, the model cannot produce a probability from that N-gram. A backoff or smoothing method is needed to handle this situation.

6. Prediction accuracy: Prediction accuracy can be calculated as:

Accuracy = Correct next-word predictions / Total test cases × 100.

For example, if 8 out of 10 test sentences have the correct next word in the selected prediction criterion, accuracy = 8/10 × 100 = 80%. Actual accuracy depends on the test corpus and evaluation method.

7. Limitations: An unsmoothed N-gram model suffers from zero probabilities for unseen N-grams, data sparsity, limited context, and weak handling of long-distance dependencies. It also requires more memory as N increases. Therefore, it is simple and fast but less reliable for real-world language prediction.

Task 2 – Backoff and Interpolation for Language Prediction

Design and implement a Python-based enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model. The system should construct unigram, bigram, and trigram models from an English corpus and accept an incomplete sentence such as:

"Machine learning can"

When the required trigram is unavailable, the system should use a backoff strategy by first checking the trigram model, then the bigram model, and finally the unigram model. Extend the system using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using predefined interpolation weights.

The system should generate the top-5 next-word predictions using:

- **Unsmoothed N-gram model**
- **Backoff model**
- **Deleted Interpolation model**

Compare the three approaches in terms of zero probabilities, prediction coverage, and prediction accuracy, and explain why backoff and interpolation are useful for sparse language data.

1. Objective: The enhanced system solves the zero-probability problem of the unsmoothed model. It first attempts to use the most specific available context and then uses shorter contexts when the required N-gram is missing.

2. Backoff strategy: For “Machine learning can”, the system first checks the trigram context (machine, learning, can). If no suitable continuation is found, it backs off to the bigram context (learning, can). If that is also unavailable, it uses the unigram model. This gives wider prediction coverage.

3. Backoff and interpolation code: ```python

```
from collections import Counter

def unigram_prob(word, uni, total):
    return uni[word] / total if total else 0

def bigram_prob(a, b, bi, uni):
    return bi[(a,b)] / uni[a] if uni[a] else 0

def trigram_prob(a, b, c, tri, bi):
    return tri[(a,b,c)] / bi[(a,b)] if bi[(a,b)] else 0

def backoff(a, b, candidates, uni, bi, tri):
    scores = {}
    for w in candidates:
        p3 = trigram_prob(a,b,w,tri,bi)
        if p3 > 0:
            scores[w] = p3
        else:
            p2 = bigram_prob(b,w,bi,uni)
            scores[w] = p2 if p2 > 0 else unigram_prob(w,uni,sum(uni.values()))
    return sorted(scores.items(), key=lambda x:x[1], reverse=True)[:5]

# Deleted-interpolation style combination
def interpolated(a,b,w,uni,bi,tri):
    total = sum(uni.values())
    p1 = unigram_prob(w,uni,total)
    p2 = bigram_prob(b,w,bi,uni)
    p3 = trigram_prob(a,b,w,tri,bi)
    return 0.2*p1 + 0.3*p2 + 0.5*p3
```

4. Deleted Interpolation: Deleted Interpolation combines different N-gram probabilities instead of depending on only one model. A typical predefined combination is:

$P(w|a,b) = \lambda1P(w) + \lambda2P(w|b) + \lambda3P(w|a,b)$, where $\lambda1 + \lambda2 + \lambda3 = 1$.

For example, $\lambda1 = 0.2$, $\lambda2 = 0.3$ and $\lambda3 = 0.5$. The highest weight is given to the trigram because it contains the most context.

5. Comparison: Unsmoothed model: simple and fast, but produces many zero probabilities for unseen N-grams. Backoff model: uses a shorter context when a longer context is unavailable, so coverage improves. Interpolation model: combines all available context levels and generally gives more stable probabilities. Exact prediction accuracy must be measured on the selected test corpus.

6. Importance for sparse data: Natural language has a very large vocabulary, so many valid word combinations will be absent from a training corpus. Backoff and interpolation reduce the effect of sparsity by using evidence from shorter contexts. They therefore improve prediction coverage and avoid assigning zero probability too easily.

Task 3 – Entropy-Based Evaluation of Language Models

Design and implement a Python program for measuring the uncertainty of N-gram language models using entropy. Given separate training and testing corpora, construct unigram, bigram, and trigram language models and calculate the probabilities assigned to words in the test sentences. For each test sentence, calculate its entropy and classify the sentence as having relatively high or low predictive uncertainty.

For example, compare sentences such as:

"The cat sits on the mat."

and

"The quantum processor redesigned the..."

The system should evaluate how predictable the next word is based on the trained language model. The program should also compare entropy values obtained using:

- **Unigram model**
- **Bigram model**
- **Trigram model**
- **Smoothed model**

Finally, interpret the results and explain how entropy can be used to measure prediction uncertainty and evaluate the reliability of a language model.

1. Objective: Entropy measures the uncertainty of a language model when predicting the words of a test sentence. Lower entropy means that the model finds the sentence more predictable, while higher entropy means greater uncertainty.

2. Probability and entropy: For a sentence containing words w1, w2, …, wn, cross-entropy can be calculated as:

$H = -(1/n) \Sigma \log_2 P(wi \mid context)$.

If a probability is zero, the logarithm is undefined, so smoothing is required for unseen events.

3. Python implementation: ```python

```
import math

def sentence_entropy(probabilities):
    values = [p for p in probabilities if p > 0]
    if not values:
        return float("inf")
    return -sum(math.log2(p) for p in values) / len(values)

# Example probabilities obtained from a trained model
cat_sentence = [0.8, 0.7, 0.6, 0.7, 0.8]
quantum_sentence = [0.5, 0.2, 0.1, 0.05, 0.1]

print("Cat sentence entropy:",
      sentence_entropy(cat_sentence))
print("Quantum sentence entropy:",
      sentence_entropy(quantum_sentence))
```

4. Interpretation of the example: “The cat sits on the mat.” contains common words and a familiar pattern, so a trained English model is likely to assign relatively higher probabilities to its words. “The quantum processor redesigned the...” may contain less frequent combinations, so the model can assign lower probabilities. Therefore, the second sentence is expected to have higher predictive uncertainty.

5. Unigram, bigram, trigram and smoothed models: A unigram model ignores word context and generally has higher uncertainty. A bigram model uses one previous word and normally gives better contextual prediction. A trigram uses two previous words and can capture more local structure. A smoothed model avoids zero probabilities for unseen N-grams and therefore gives more robust entropy estimates on unseen test data.

6. High and low uncertainty: Low entropy indicates that the next words are relatively predictable from the training corpus. High entropy indicates that many continuations are possible or that the sentence contains unfamiliar word combinations. Entropy therefore provides a numerical measure for comparing language-model uncertainty.

7. Reliability: A reliable language model should assign reasonably high probability to likely test sequences and should not collapse when an unseen N-gram occurs. Comparing entropy across models helps identify whether additional context or smoothing improves predictive performance. Lower test cross-entropy generally indicates better probabilistic prediction, provided the models are evaluated on the same test set.

Task 4 – Comparative POS Tagging System

Design and implement a Python-based POS tagging system that assigns grammatical tags to words in English sentences.

The system should implement three approaches:

1. Rule-Based POS Tagging

Use a combination of lexical dictionaries, word patterns, and grammatical rules to assign POS tags.

2. Stochastic POS Tagging

Construct a statistical POS tagger using:

- **Word/tag frequencies**
- **Tag transition probabilities**
- **Probabilities obtained from a training corpus**

3. Transformation-Based Tagging

Initially assign tags using a simple tagging strategy and subsequently apply transformation rules to correct likely tagging errors.

For example:

"I book a ticket."

and

"I read a book."

The system should demonstrate how the same word can receive different POS tags depending on its context. Use an appropriate English corpus such as the Brown Corpus or another publicly available tagged corpus. The program should accept user-entered sentences, display the POS tags produced by all three approaches, and compare their results using suitable evaluation measures such as tagging accuracy. Finally, analyze the differences among the three approaches and explain the advantages and limitations of rule-based, stochastic, and transformation-based POS tagging.

1. Objective: The system assigns a grammatical category to every word and compares rule-based, stochastic, and transformation-based POS tagging. The important point is that the same word can receive different tags depending on its context.

2. Rule-based tagging: A rule-based tagger uses a dictionary, word patterns, and grammar rules. For example, a word ending in "-ing" may be tagged as VBG, while a known determiner such as "the" is tagged DT. Rules are transparent and easy to understand but may fail when context is ambiguous.

3. Stochastic tagging: A stochastic tagger learns probabilities from a tagged corpus. It can use word/tag frequencies and transition probabilities such as P(tag_i | tag_{i-1}). The most probable tag sequence is

selected for the input sentence. This approach can handle contextual ambiguity better when sufficient training data is available.

4. Transformation-based tagging: Transformation-based tagging first assigns simple initial tags and then applies learned correction rules. For example, a rule may change a noun tag to a verb tag when a word occurs after a pronoun in a suitable context. The approach combines an initial tagging strategy with contextual error correction.

5. Example: context-dependent word: Sentence 1: “I book a ticket.”

Here “book” is a verb (VB) because it describes the action performed by “I”.

Sentence 2: “I read a book.”

Here “book” is a noun (NN) because it is the object of the verb “read”.

Thus, lexical information alone is not always enough; context is required.

6. Python demonstration: ```python

```
import nltk
from nltk import word_tokenize, pos_tag

sentences = [
    "I book a ticket.",
    "I read a book."
]

for sentence in sentences:
    tokens = word_tokenize(sentence)
    print(sentence)
    print(pos_tag(tokens))
```

7. Evaluation: Tagging accuracy is calculated as:

Accuracy = Correctly tagged words / Total words × 100.

For example, if 92 out of 100 words receive the correct Penn Treebank tag, accuracy is 92%. The same test set should be used for all three approaches for a fair comparison.

8. Comparison: Rule-based tagging is interpretable and works well for clearly defined patterns, but it needs many manually designed rules. Stochastic tagging learns from data and handles contextual ambiguity better, but its performance depends on training data and probabilities. Transformation-based tagging can correct systematic errors and is interpretable, but it requires useful transformation rules or learned transformations. A suitable tagged corpus such as the Brown Corpus can be used for training and evaluation as required by the task.

9. Overall conclusion: The three approaches solve POS tagging in different ways. Rule-based systems rely on linguistic knowledge, stochastic systems rely on statistical evidence, and transformation-based systems improve an initial tagging through corrections. Context is essential for words such as “book”, so a good POS tagger should consider both lexical information and surrounding tags/words.

Answer Note: The question paper requires execution on a suitable corpus. Where dataset-specific accuracy or entropy values are requested, the answers provide the correct calculation method and illustrative examples; exact experimental values depend on the corpus and execution environment.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25%	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30%	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15%	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10%	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1	3	3	3	3	4		75
2	3	4	3	3	3		
3	3	3	4	3	3		
4	3	3	3	4	3		

Faculty In-charge	Course Coordinator	Program Director
Dr. T. Kumaragurubaran		
Signature:	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____